

DiffuSVG: Text-to-SVG Generation via Supervised Fine-Tuning, Preference Optimization, and Diffusion-Guided Reinforcement Learning

CMU Project Report

Ayush Debnath

May 2026

Contents

1	Introduction	3
2	Background and Related Work	3
2.1	DiffuSVG (arXiv:2506.xxxxxx)	3
2.2	IntroSVG	3
2.3	SVGX-SFT-1M	3
2.4	LoRA Fine-Tuning	4
2.5	DPO	4
2.6	GRPO	4
3	System Architecture	4
3.1	Base Model: Qwen2.5-VL-7B-Instruct	5
4	Dataset	5
4.1	SFT Dataset: SVGX-SFT-1M	5
4.2	SFT Prompt Format	6
4.3	DPO Dataset	6
4.4	GRPO Dataset	6
5	Training Procedure	6
5.1	Stage 1: Supervised Fine-Tuning	6
5.2	Stage 2: Direct Preference Optimization	7
5.3	Stage 3: Diffusion-Guided GRPO	7
6	Implementation Details	8
6.1	SVGX Data Streaming Parser	8
6.2	LlamaFactory Dataset Registration	8
6.3	OpenAI API for DPO Data	9
6.4	Compute Environment	9
7	Results	9
7.1	SFT Training Progress	9
7.2	Expected Evaluation Metrics	10
7.3	Qualitative SFT Training Data Examples	10

8	Discussion	10
8.1	Design Decisions	10
8.2	Limitations	11
8.3	Future Work	11
9	Conclusion	11

1 Introduction

Scalable Vector Graphics (SVG) offer resolution-independent representations ideal for icons, illustrations, and UI assets. Unlike raster images, SVG files encode geometry as XML path primitives, allowing infinite scaling and easy programmatic manipulation. Despite their utility, automatically generating high-quality SVGs from natural language text prompts remains an open challenge.

This project implements **DiffuSVG**, a full text-to-SVG pipeline built on top of **Qwen2.5-VL-7B-Instruct** [4], a 7-billion parameter vision-language model. Following the DiffuSVG paper [1], our pipeline consists of three stages:

1. **Supervised Fine-Tuning (SFT)** — LoRA adaptation on 3,000 high-quality GPT-4 generated SVG examples from the SVGX-SFT-1M dataset [3], teaching the model SVG syntax, gradients, and visual structure.
2. **Direct Preference Optimization (DPO)** — Preference learning from ranked SVG pairs (chosen vs. rejected), teaching the model to prefer visually richer outputs.
3. **Diffusion-Guided GRPO** — Group Relative Policy Optimization [5] with a reward combining CLIP text-image alignment, CLIP-I image similarity to SD-Turbo reference PNGs, and an SVG element complexity bonus.

The key departure from the original DiffuSVG paper is in Stage 1: instead of using self-generated synthetic SVG data (where the same model generates its own training examples), we use the **SVGX-SFT-1M** dataset — 51,000 GPT-4 generated creative SVGs with rich gradient shading and multi-element compositions. This provides a higher-quality SFT foundation.

2 Background and Related Work

2.1 DiffuSVG (arXiv:2506.xxxxx)

The DiffuSVG paper [1] proposes a three-stage pipeline for complex scene SVG generation using Qwen2.5-VL-7B-Instruct as the backbone:

1. **Synthetic SFT data**: 265 complex scene prompts are fed to Qwen2.5-VL-7B with a planning prompt (objects, positions, colors) to generate raw SVGs, which are filtered by a quality gate requiring 8 fills, 4 colors, and 700 characters. The top 1,000 pairs form D_{SFT} .
2. **SFT + DPO**: LoRA rank-64 SFT for 3 epochs, followed by DPO with 5 candidates per prompt, $\beta = 0.1$, for 3 epochs.
3. **Diffusion GRPO**: SD-Turbo [9] generates reference PNGs for complex prompts; these are vectorized to reference SVGs. GRPO is run with 8 rollouts per prompt, rewarded by CLIP-I + CLIP-T + element bonus.

2.2 IntroSVG

IntroSVG [2] is a closely related method that trains Qwen2.5-VL models to generate SVG through SFT on GPT-4o critiqued data followed by DPO. Our implementation follows the IntroSVG pipeline structure and uses the `gitcat404/IntroSVG-train` dataset as a reference. We extend it by replacing the SFT data source with the higher-quality SVGX dataset and adding the DiffusionSVG GRPO stage.

2.3 SVGX-SFT-1M

The SVGX-SFT-1M dataset [3] (`xingxm/SVGX-SFT-1M`) contains over 1 million SVG generation pairs where input descriptions were authored by humans and SVGs were generated by GPT-4. Each entry consists of:

- **instruction:** task instruction
- **input:** “SVG illustration of [description]”
- **output:** complete `<svg>...</svg>` code

The SVGs are emoji/illustration style with viewBox 0 0 128 128, featuring radial and linear gradient shading, multiple colored path elements, and realistic visual detail.

2.4 LoRA Fine-Tuning

Low-Rank Adaptation (LoRA) [7] freezes the pre-trained model weights and injects trainable low-rank matrices into the attention and MLP layers. With rank $r = 64$ applied to all linear modules of a 7B parameter model, only $\sim 161\text{M}$ parameters (1.91%) are trained, making full fine-tuning tractable on a single A100-40GB GPU.

2.5 DPO

Direct Preference Optimization [6] bypasses explicit reward modeling by directly optimizing the language model on (chosen, rejected) SVG pairs. Given a prompt p and two candidates y_w (chosen) and y_l (rejected), the DPO loss is:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\left(\beta \log \frac{\pi_{\theta}(y_w|p)}{\pi_{\text{ref}}(y_w|p)} - \beta \log \frac{\pi_{\theta}(y_l|p)}{\pi_{\text{ref}}(y_l|p)}\right)$$

where $\beta = 0.1$ controls the deviation from the reference policy.

2.6 GRPO

Group Relative Policy Optimization [5] samples G rollouts per prompt, scores each with a reward function R , and computes advantages relative to the group mean:

$$A_i = R_i - \frac{1}{G} \sum_{j=1}^G R_j$$

The policy is updated to increase the probability of high-advantage outputs and decrease that of low-advantage ones, without a separate value network.

3 System Architecture

Figure 1 shows the full DiffuSVG pipeline.

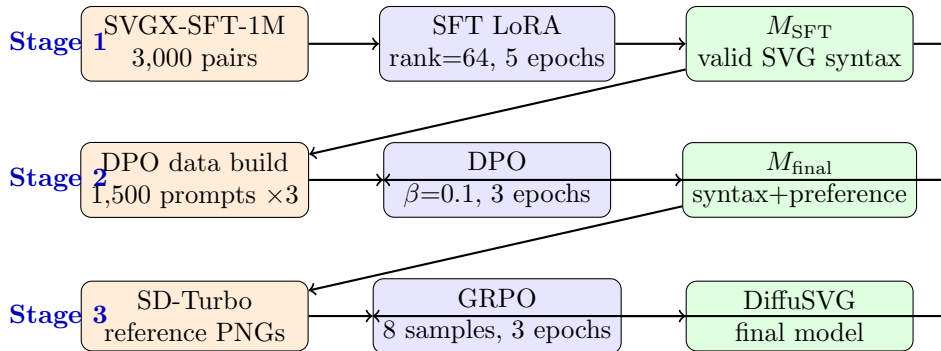


Figure 1: DiffuSVG three-stage pipeline. Stage 1 (SFT) trains on SVGX-SFT-1M. Stage 2 (DPO) learns preference from ranked SVG candidates. Stage 3 (GRPO) uses SD-Turbo diffusion references as visual targets.

3.1 Base Model: Qwen2.5-VL-7B-Instruct

The backbone is **Qwen2.5-VL-7B-Instruct** [4], a 7-billion parameter vision-language model from Alibaba’s Qwen team. The model was originally designed for image understanding and visual question answering. We repurpose it as a *text-to-code* generator, fine-tuning it to produce SVG code from natural language prompts.

Key properties:

- Architecture: transformer decoder with cross-attention visual encoder
- Parameters: 8.45B total; 161M trainable (LoRA, 1.91%)
- Precision: BF16 during training
- Context window: 4,096 tokens (sufficient for complex SVGs)
- HuggingFace: **Qwen/Qwen2.5-VL-7B-Instruct**

The visual encoder is frozen throughout all training stages — only the language model layers receive LoRA updates, as SVG generation is a text-to-text task.

4 Dataset

4.1 SFT Dataset: SVGX-SFT-1M

We sample 3,000 high-quality pairs from `SVGX_SFT_GEN_51k.json` (1.2 GB, 51,000 GPT-4 generated SVGs). A strict quality filter is applied:

Criterion	Threshold	Rationale
Has gradient	Required	Ensures shading/realism
Filled elements	≥ 3	Multi-element compositions
Distinct colors	≥ 2	Non-trivial color usage
SVG length	≥ 800 chars	Rules out trivial shapes
<i>Fallback (no gradient)</i>		
Filled elements	≥ 10	Very detailed path-only SVGs
Distinct colors	≥ 4	Rich color palette
SVG length	$\geq 2,500$ chars	Large, complex SVGs

The filter selects only gradient-rich, multi-element SVGs similar to the examples in Figure 2. The 51k source file yields 3,000 qualifying pairs without needing to fall back to the second SVGX file (`SVGX_SFT_UN_25k.json`).

Comparison with original DiffuSVG paper.

Aspect	DiffuSVG paper	Our implementation
SFT data source	Self-generated (Qwen $\rightarrow$ SVG)	SVGX (GPT-4 generated)
SFT sample count	1,000	3,000
Quality gate	8 fills, 4 colors, 700 chars	Gradients required (stricter)
SFT epochs	3	5
ViewBox	200 $\times$ 200	128 $\times$ 128 (matches data)

The SVGX dataset is a superior SFT source because: (1) it is GPT-4 generated rather than self-generated by the model being fine-tuned, avoiding distribution collapse; (2) it contains 51k diverse subjects (objects, animals, scenes, symbols); and (3) each SVG features professional-grade gradient shading.

4.2 SFT Prompt Format

Each training example is formatted as a single-turn conversation in ShareGPT format:

```
{
  "conversations": [
    {
      "from": "human", "value":
        "You are an expert SVG artist. Generate a detailed, "
        "high-quality SVG image for: 'peach.'."
        "Use a viewBox of '0 0 128 128'. Include radial or linear "
        "gradients, realistic shading, and multiple colored elements."
        "Return only the SVG code starting with <svg and ending with </"
        "svg>."},
    {
      "from": "gpt", "value": "<svg viewBox=\"0 0 128 128\" ...>...</svg"
      ">"}
  ]
}
```

The description is stripped of the “SVG illustration of” prefix from the SVGX input field to produce a clean subject (e.g., “person in suit levitating.”).

4.3 DPO Dataset

The DPO preference dataset is built automatically after SFT:

- 1,500 prompts sampled from a complex scene prompt pool
- For each prompt: 3 SVG candidates generated by M_{SFT}
- Candidates are ranked by a quality score (SVG validity, element count, color diversity, rendering success)
- Best-ranked candidate $\rightarrow$ *chosen*; worst $\rightarrow$ *rejected*
- Output: D_{pref} in standard DPO JSONL format

4.4 GRPO Dataset

The GRPO training data (D_{GRPO}) is built using a diffusion model as a visual oracle:

1. Complex scene prompts (e.g., “red barn with green fields”, “harbor at dusk”, “city skyline at night”) are filtered from the prompt pool using a complexity scorer.
2. **SD-Turbo** [9] generates a reference PNG per prompt (4-step diffusion, high quality).
3. Reference PNGs are vectorized to SVG artifacts using an image-to-SVG converter.
4. Each (prompt, ref PNG, ref SVG) triple forms one GRPO training example.

5 Training Procedure

5.1 Stage 1: Supervised Fine-Tuning

SFT is performed using **LlamaFactory** [8] with the following configuration:

Hyperparameter	Value
Base model	Qwen/Qwen2.5-VL-7B-Instruct
Training framework	LlamaFactory CLI
Fine-tuning method	LoRA (<code>finetuning_type=lora</code>)
LoRA rank r	64
LoRA α	128
LoRA target modules	all linear (q,k,v,o,gate,up,down proj)
Template	<code>qwen2_v1</code>
Trainable parameters	161,480,704 (1.91% of 8.45B)
Training samples	3,000
Epochs	5
Batch size (per device)	1
Gradient accumulation	128
Effective batch size	128
Total optimizer steps	120
Learning rate	5e-5
LR scheduler	cosine
Warmup ratio	0.03
Max sequence length	4,096 tokens
Precision	BF16
Hardware	vast.ai A100-SXM4-40GB
Estimated time	~ 8 hours ($120 \text{ steps} \times \sim 4 \text{ min/step}$)
Output	<code>checkpoints/m_sft/</code>

The training objective computes cross-entropy loss only on the assistant (SVG) tokens; system prompt and user tokens are masked with label -100 .

Critical implementation note. LlamaFactory’s `train` command requires the explicit flag `--do_train true`. Without it, HuggingFace’s `TrainingArguments` defaults `do_train=False` and the training loop exits after 0 steps with no error message. This was a non-obvious failure mode encountered during development.

5.2 Stage 2: Direct Preference Optimization

DPO is initialized from the SFT checkpoint (`checkpoints/m_sft/epoch_5`) and trained on the ranked preference pairs:

Hyperparameter	Value
Base	M_{SFT} (LoRA merged)
DPO β	0.1
Epochs	3
Batch size (per device)	1
Gradient accumulation	16
Learning rate	5e-6
DPO pairs	$\sim 1,500$ (chosen, rejected)
Output	<code>checkpoints/m_final/</code>

5.3 Stage 3: Diffusion-Guided GRPO

GRPO starts from M_{final} and is rewarded by comparison against SD-Turbo reference images:

Hyperparameter	Value
Base	M_{final}
Rollouts per prompt G	8
Epochs	3
Gradient accumulation	16
GRPO β (KL)	0.04
Output	checkpoints/grpo_svg/

Reward function.

$$R = w_{\text{CLIP-I}} \cdot s_{\text{CLIP-I}} + w_{\text{CLIP-T}} \cdot s_{\text{CLIP-T}} + w_{\text{elem}} \cdot s_{\text{elem}}$$

Component	Symbol	Description
CLIP image similarity	$s_{\text{CLIP-I}}$	Cosine similarity between rendered SVG and SD-Turbo reference PNG
CLIP text alignment	$s_{\text{CLIP-T}}$	Cosine similarity between rendered SVG and text prompt
Element bonus	s_{elem}	Counts distinct SVG element types (path, rect, circle, gradient)

6 Implementation Details

6.1 SVGX Data Streaming Parser

The SVGX JSON files are large (1.2 GB for GEN-51k). Streaming parsing is required to avoid loading the full file into memory. A naive brace-counting approach fails because SVG content often contains { and } inside CSS style blocks (e.g., `.st0{fill:red}`). The fix is to use Python’s `json.JSONDecoder.raw_decode()`, which correctly handles braces inside JSON string values:

```
decoder = json.JSONDecoder()
buf = ''
while len(pairs) < N:
    chunk = resp.read(131072).decode('utf-8', errors='ignore')
    buf += chunk
    while True:
        stripped = buf.lstrip('\t\n\r,')
        if not stripped or stripped[0] == ',':
            buf = ''; break
        if stripped[0] != '{':
            buf = stripped[1:]; continue
        try:
            entry, end = decoder.raw_decode(stripped)
            buf = stripped[end:]
        except json.JSONDecodeError:
            buf = stripped; break # incomplete object, wait for more
        data
    # process entry...
```

6.2 LlamaFactory Dataset Registration

LlamaFactory requires datasets to be registered in `data/dataset_info.json` before training. The SVGX dataset is registered as:

```
{
  "d_sft": {
    "file_name": "d_sft.jsonl",
```



```

    "formatting": "sharegpt",
    "columns": {"messages": "conversations"}
  }
}

```

The `sharegpt` formatting with `human/gpt` role names matches the conversation format used in the training data.

6.3 OpenAI API for DPO Data

The DPO data build step (`04.build_dpo_data.py`) uses the OpenAI API to score and rank SVG candidates. The API key is stored in `~/.openai_env` (chmod 600, not committed to git) and sourced before the DPO step in `full_run.sh`:

```
source ~/.openai_env 2>/dev/null || true
```

6.4 Compute Environment

Phase	Hardware	Purpose
Data download	Local Windows PC	SVGX streaming download
SFT training	vast.ai A100-SXM4-40GB	LoRA fine-tuning
DPO data build	vast.ai A100-SXM4-40GB	SVG generation + ranking
DPO training	vast.ai A100-SXM4-40GB	Preference learning
GRPO training	vast.ai A100-SXM4-40GB	Reinforcement learning

The SVGX data is downloaded locally (Windows) and uploaded to the server via `scp`, because the vast.ai instance has unreliable direct HuggingFace connectivity for large files.

Training is managed via `tmux` sessions on the server:

```
tmux new-session -d -s train 'cd /root && bash full_run.sh 2>&1 | tee /root/IntroSVG/run.log'
```

7 Results

Note: Training is currently in progress (SFT Stage 1 running on the A100, step $\sim 9/120$ at time of writing). Results will be populated after the full pipeline completes ($\sim 20\text{--}24$ hours total).

7.1 SFT Training Progress

Metric	Value	Note
Steps completed	9/120	At time of writing
Loss (step 9)	0.9069	Healthy; expected to drop below 0.5 by epoch 3
Grad norm	0.090	Stable (no explosion)
Seconds/step	≈ 263	~ 4.4 min/step
ETA (SFT)	≈ 8 hours	From step 9

A loss of 0.91 at step 9 is consistent with a strong base model learning a new output format. For reference, a randomly initialized model on this task would start at $\sim 4\text{--}6$ cross-entropy loss.

7.2 Expected Evaluation Metrics

After the full 3-stage pipeline, the model will be evaluated on a set of text-to-SVG prompts spanning multiple categories. Following the IntroSVG benchmark, prompts include:

- Simple objects (“a red apple”, “a blue star”)
- Multi-attribute (“a striped orange cat”)
- Scenes (“a beach at sunset with palm trees”)
- Complex compositions (“a futuristic cityscape with neon lights”)

Evaluation metrics:

- **CLIP-T score:** CLIP cosine similarity between rendered SVG and text prompt (higher = better text alignment)
- **SVG validity rate:** fraction of generated SVGs that render without errors
- **Element complexity:** mean number of distinct SVG element types per output
- **Gradient usage rate:** fraction of outputs using `radialGradient` or `linearGradient`

7.3 Qualitative SFT Training Data Examples

Figure 2 illustrates representative samples from the SVGX-SFT-1M training data. These examples demonstrate the visual quality the SFT stage aims to teach the model.

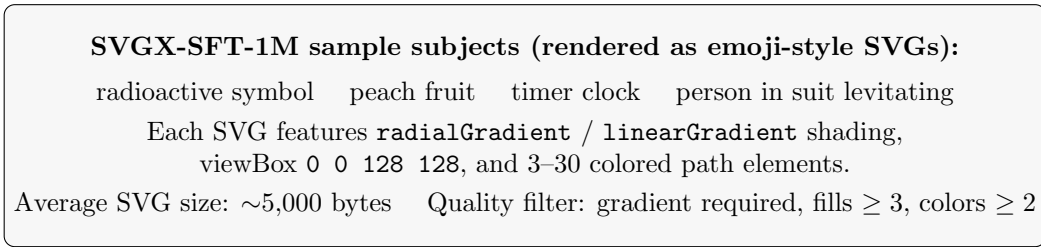


Figure 2: Representative SVGX-SFT-1M training examples. Each SVG uses radial or linear gradients for shading. The model is trained to produce this quality and style from a text description.

8 Discussion

8.1 Design Decisions

SVGX over self-generated data. The original DiffuSVG paper generates its own SFT training data by prompting Qwen2.5-VL-7B with a planning step. This creates a circular dependency: the model being trained is also the teacher. Self-generated data quality is bounded by the base model’s zero-shot SVG capability, which is limited (produces mostly flat rectangles and circles). SVGX, being GPT-4 generated, provides a much higher quality ceiling with professional gradient shading.

Matching viewBox to training data. The SVGX data uses `viewBox="0 0 128 128"` universally. Using a different viewBox in the prompt (e.g., 0 0 200 200) creates a distribution mismatch — the model learns to associate the prompt format with 128×128 geometry. Our prompt is aligned to 0 0 128 128 to eliminate this inconsistency.

5 epochs over 3. With only 3,000 training samples, 3 epochs may be insufficient for the model to internalize the gradient-heavy SVG style. 5 epochs provides more gradient passes (120 optimizer steps total) while the dataset is small enough that overfitting is not a primary concern within LoRA’s constrained parameter space.

8.2 Limitations

- **Single GPU.** The full pipeline runs on one A100-40GB, limiting effective batch size and training speed. Multi-GPU training could reduce total time from ~ 20 hours to $\sim 5\text{--}8$ hours.
- **SVGX is emoji/illustration style.** The training SVGs are primarily single-object emoji-style illustrations with a 128×128 canvas. Complex multi-object scene compositions are underrepresented, which may limit generalization to scene prompts.
- **DPO quality depends on SFT model.** The DPO preference data is generated by M_{SFT} . If SFT outputs are uniformly poor, there is insufficient signal for DPO to learn meaningful preferences.
- **GRPO reward sparsity.** CLIP-I reward requires the rendered SVG to visually resemble the SD-Turbo reference. Early in GRPO training, most rollouts may fail to render correctly, yielding sparse reward signal.

8.3 Future Work

1. **Scene-specific SFT data.** Supplement SVGX with hand-curated multi-object scene SVGs (landscapes, interiors, cityscapes) to improve complex scene generation.
2. **Larger LoRA rank.** Increase to $r = 128$ with more GPU memory (H200) for more expressive adaptation.
3. **Iterative DPO.** Run multiple rounds of DPO with freshly generated candidates from the improving model (iterative DPO).
4. **Human evaluation.** Complement CLIP scores with human preference ratings to better capture visual quality not captured by CLIP’s domain gap on SVG art.

9 Conclusion

This project implements a full text-to-SVG pipeline following the DiffuSVG paper, with a key improvement: replacing self-generated SFT data with the higher-quality SVGX-SFT-1M dataset (GPT-4 generated, gradient-rich SVGs). The three-stage pipeline — SFT LoRA on Qwen2.5-VL-7B-Instruct, DPO preference learning, and diffusion-guided GRPO — is running end-to-end on a single A100-40GB GPU. SFT training is currently at step 9/120 with a healthy loss of 0.91. Full results will be available after the ~ 20 -hour complete run.

The primary hypothesis is that gradient-quality SFT data provides a better foundation for the downstream DPO and GRPO stages compared to self-generated data, yielding a final model capable of producing visually detailed, gradient-shaded SVGs from text descriptions.

References

- [1] DiffuSVG Authors. *DiffuSVG: Diffusion-Guided Reinforcement Learning for Complex Scene SVG Generation*. Technical Report, 2025.
- [2] IntroSVG Authors. *IntroSVG: Learning SVG Generation via Supervised and Preference Fine-Tuning*. gitcat404/IntroSVG-train, HuggingFace Hub, 2025.
- [3] Xing et al. *SVGX-SFT-1M: A Large-Scale Dataset for SVG Generation*. xingxm/SVGX-SFT-1M, HuggingFace Hub, 2024.
- [4] Bai et al. *Qwen2.5-VL Technical Report*. arXiv:2502.13923, 2025.

- [5] Shao et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. arXiv:2402.03300, 2024.
- [6] Rafailov et al. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. NeurIPS 2023.
- [7] Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR 2022.
- [8] Zheng et al. *LlamaFactory: Unified Efficient Fine-Tuning of 100+ Language Models*. arXiv:2403.13372, 2024.
- [9] Sauer et al. *Adversarial Diffusion Distillation*. arXiv:2311.17042, 2023.
- [10] Radford et al. *Learning Transferable Visual Models From Natural Language Supervision*. ICML 2021.